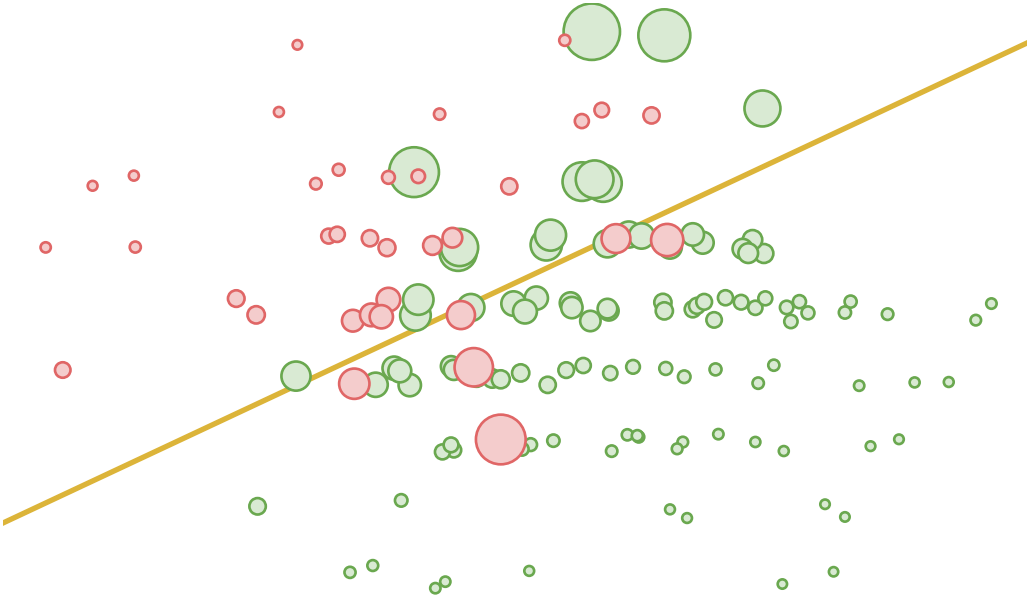


UNIT 01 · CHAPTER 2

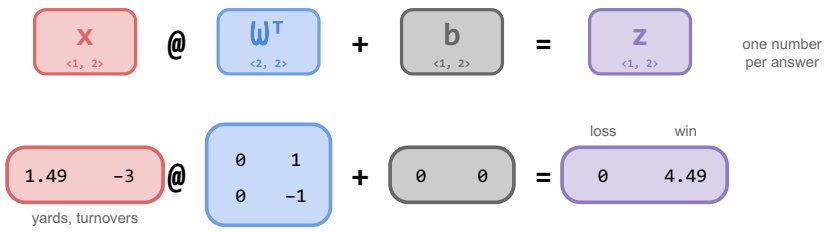
Loss Functions

CSE 10124 · Building ChatGPT
William Theisen · University of Notre Dame

Predictions as Vectors

In the last chapter every Notre Dame game became a vector, and a matrix of weights (plus a bias) turned it into a new vector, a **linear transformation** that is already a model. That chapter ended needing a single number that says how wrong the model is, and to build one we first have to decide what its new vector means.

Let's describe each game with two numbers, Notre Dame's **yard difference** (in hundreds of yards) and its **turnover difference**, and ask the question we keep coming back to: **"Did we win?"** In `2012` Notre Dame beat Navy `50` to `10`, gaining `149` more yards and turning the ball over `3` fewer times, so that game is **x** = `[1.49, -3]`.



The weights here are ones we picked by hand. Like PyTorch's linear layer, **W** has one row per answer and is flipped before multiplying (**W**^{T}). The row for "loss" is all zeros, and the row for "win" adds the yards and subtracts the turnovers. Out come two numbers, one for each answer the model could give, and for Navy the "win" number is a lot bigger (`4.49` to `0`).

Classes

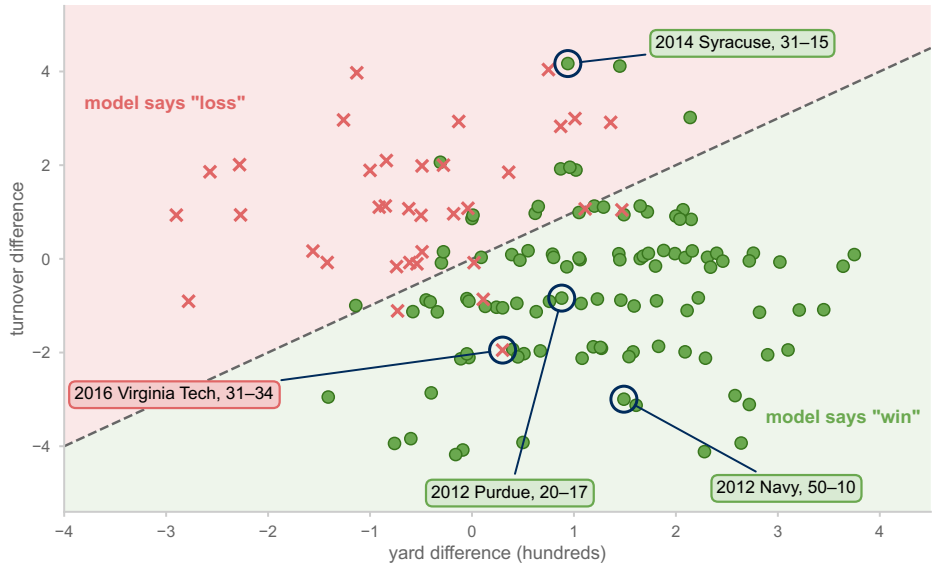
Notice that the model doesn't give us an answer. **It gives us a vector, with one number for every answer it could have given**, and we still have to decide what those numbers mean.

The possible answers to a question like ours are called **classes** (or targets). "Did we win?" has two, loss (index `0`) and win (index `1`), so our output vector is `<1, 2>`. The digit classifier from lecture has `10` classes, one per digit, and a language model has one class for every piece of text it knows.

To turn the output vector into an answer we take the index of its biggest number. That index is the model's **predicted class**, the answer it actually gives.

The Predicted Class

For our hand-picked weights, "biggest number" means "win" whenever the yard difference (in hundreds) is bigger than the turnover difference, which draws a line through every game:



Every game, 2012 to 2022: green dots are wins and red crosses losses (nudged up and down a little so games with the same turnover difference don't hide each other).

Taking the biggest number, our model gets `121` of the `141` games right (`86%`), which isn't bad for two weights we made up. It does lose something on the way, though. Navy (`z = [0, 4.49]`) and a `20` to `17` win over Purdue that same year (`z = [0, 1.88]`) both come out as just "win", even though the model's numbers were very different.

Logits

Here is our model on the first three games of `2012` in PyTorch. `W` is **W** (one row per class), and `@` is matrix multiplication, as before:

```
import torch
import pandas as pd

games = pd.read_csv("nd_football.csv")
cols = ["yard_diff", "turnover_diff"]
X = torch.tensor(games[cols].values,
                 dtype=torch.float32)
X[:, 0] = X[:, 0] / 100     # yards, in hundreds
W = torch.tensor([[0., 0.],   # loss
                  [1., -1.]]) # win
b = torch.zeros(2)
logits = X @ W.T + b
print("logits:", logits[:3])
```

OUTPUT

```
logits: tensor([[0.0000, 4.4900],
        [0.0000, 1.8800],
        [0.0000, 1.6300]])
```

These raw outputs are called **logits**: the model's unnormalized outputs, with one number for each possible class. They can be any size and either sign. The `2012` Wake Forest game (`38` to `0`, `375` more yards) gets a "win" logit of `3.75`, and the `2018` Cotton Bowl against Clemson (`3` to `30`) gets `-3.90`.

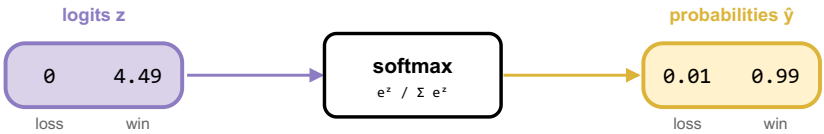
Unfortunately, that makes them hard to read. Is `1.88` sure of a win, or only slightly leaning that way? Nothing ties the numbers to any scale, so **we can't say how confident the model is**.

Softmax

To put logits on a scale we use **softmax**, a function that turns logits into nonnegative probabilities that add up to `1`. It raises e to the power of each logit (so every number becomes positive) and then divides each one by their total (so they sum to `1`):

one probability per class

[2.1] $$\hat{y}_i = \text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$



For Navy, e^0 = `1` and $e^{4.49}$ = `89.1`, so the model gives a win `89.1 / 90.1 = 0.989` and a loss `0.011`. Purdue's logits become `0.868` for a win, a good deal less sure. And the `2016` Navy game (Notre Dame gained `2` more yards, the turnovers were even, and it lost `27` to `28`) gets logits of `[0, 0.02]` and probabilities of `0.495` and `0.505`, a coin flip, which for a one-point game seems about right.

The biggest logit always becomes the biggest probability, so **softmax never changes the prediction**. It only rescales the logits so we can read them. We call the probabilities it produces $\hat{y}$, our model's **output prediction**.

The Ground Truth

Every game in our data comes with what actually happened, the **ground truth**. To compare it with $\hat{y}$ we write it in the same shape, as a **one-hot vector**: the same size as the output vector, with a `1` at the index of the correct class and `0`s everywhere else. A win is y = `[0, 1]` and a loss is y = `[1, 0]`.

Game	Score	$\hat{y}$ (loss, win)	y
`2012` Navy	`50–10`	`[0.011, 0.989]`	`[0, 1]`
`2012` Purdue	`20–17`	`[0.132, 0.868]`	`[0, 1]`
`2016` Navy	`27–28`	`[0.495, 0.505]`	`[1, 0]`
`2016` Virginia Tech	`31–34`	`[0.091, 0.909]`	`[1, 0]`

Notice that the ground truth is also a probability vector, just one with no doubt in it. We can think of it as the prediction a perfect model would have made (all of its probability on what happened).

Virginia Tech is the interesting row. Notre Dame outgained them by `30` yards and won the turnover battle by `2`, so the model was `91%` sure of a win, and Notre Dame lost by `3`. What we want now is **a single number that says how far apart $\hat{y}$ and y are**.

Right or Wrong

The simplest number we could use is the one we used to pick a class: was the biggest probability on the right class or not? Count the games where it was and we have the model's **accuracy**, `121` of `141`.

Unfortunately, right-or-wrong **throws away the probabilities**. Navy (`0.989` on a win) and Purdue (`0.868`) count as exactly the same kind of "right". Virginia Tech (`91%` sure of a win that never came) counts exactly the same as the `2016` Navy coin flip that happened to land on the wrong side.

It also barely moves. Change the weights a little and almost every game stays on the same side of the line, so the accuracy stays put. A number that ignores small changes is no help when the whole point of having a number is to tell better weights from slightly worse ones.

So we want something finer: a number that rewards being sure and right, punishes being sure and wrong, and changes (a little) whenever the probabilities do.

Loss

A **loss** is a single number that measures how well a model's prediction matches the expected answer. By convention lower is better, and a perfect prediction has a loss of `0`. The function that calculates it is called a **loss function**.

Since $\hat{y}$ and y are both vectors, a natural first try is the one we already know: the squared distance between them, adding up the squared difference in each position,

squared error

$$(\hat{y}_0 - y_0)^2 + (\hat{y}_1 - y_1)^2$$

For Navy that's $0.011^2 + 0.011^2 = 0.0002$, nearly perfect. Purdue gets `0.035`, the `2016` Navy coin flip `0.51`, and Virginia Tech $0.909^2 + 0.909^2 = 1.65$. It ranks the four games the way we would, which is a good start.

Squared error is a perfectly good loss, and it's the usual choice when a model predicts a number (a score, a temperature). When the model predicts probabilities, though, there is a loss that fits them better, and it only needs one of the two positions.

Cross-Entropy

Instead of comparing every position, look only at **the probability the model gave to what actually happened**, and take the negative of its log. For Navy that's $-\ln 0.989 = 0.011$, for Purdue $-\ln 0.868 = 0.142$, for the 2016 Navy game $-\ln 0.495 = 0.703$ and for Virginia Tech $-\ln 0.091 = 2.40$.

Our one-hot y lets us write that without saying "the right class" in words. Multiply each y_i by $\log \hat{y}_i$ and add them up: every term where $y_i = 0$ vanishes, and only the right class is left.

[2.2]

one game, summed over the classes

$$\mathcal{L}_{ce} = -\sum_i y_i \cdot \log(\hat{y}_i)$$

2016 Virginia Tech, a loss: y = [1, 0]

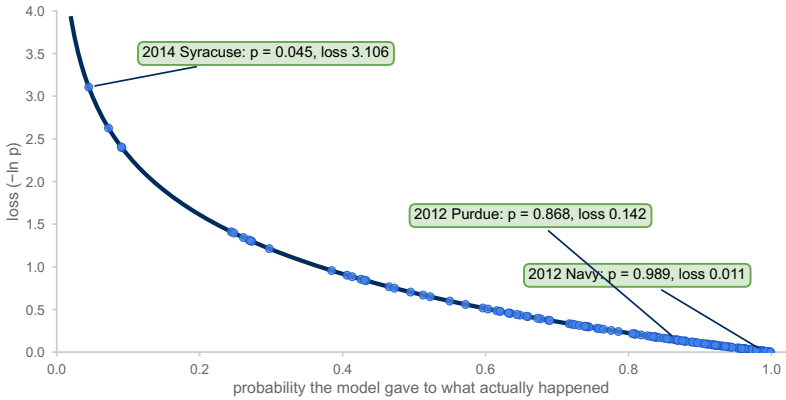
$$-(1 \cdot \log 0.091 + 0 \cdot \log 0.909) = 2.40$$

(That sum is the dot product of y and $\log \hat{y}$, from the last chapter.) This is **cross-entropy loss**, and it's one of the most common loss functions there is. It's the loss we'll use for almost everything in this course, including irishQwen, because it is **the loss used to train LLMs**.

A small note on the log: in PyTorch (and in this book) `log` means the natural log, `ln`. Other bases only scale every loss by the same amount.

Surprise

One way to read $-\ln p$ is as how surprised the model should be by what happened. If it gave the result a probability of `1`, it isn't surprised at all, $-\ln 1 = 0$. At `0.5` the surprise is `0.693`, at `0.1` it's `2.30`, at `0.01` it's `4.61`, and as the probability heads to `0` the surprise heads to infinity.

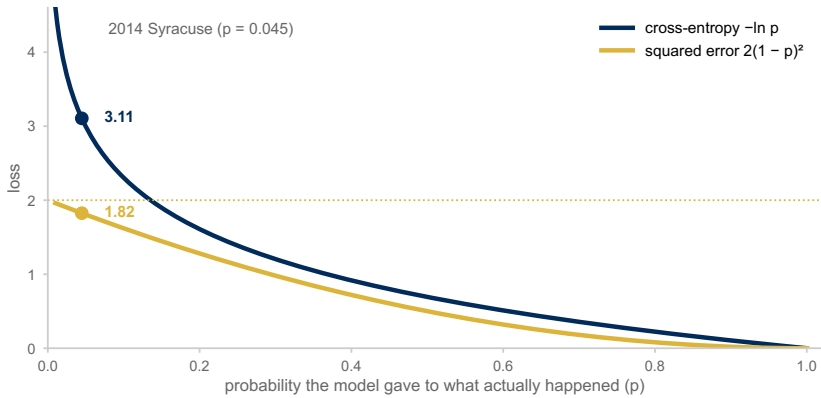


Every game's loss: each dot is one game, at the probability our model gave to its real result. Most sit at the right end with small losses, and a few sit far up the curve on the left.

The name comes from information theory, where Claude Shannon (`1948`) measured how surprising a source of messages is with its **entropy**, $-\Sigma\ p(\mathbf{x})\ \log_2 p(\mathbf{x})$. Cross-entropy is the same sum with two different distributions in it, the true one ($\mathbf{y}$) and the one the model believes ($\hat{\mathbf{y}}$). It's mostly outside the scope of this class, but it really isn't that complicated.

Confidently Wrong

The left end of that curve is where cross-entropy earns its keep. In `2014` Notre Dame beat Syracuse `31` to `15` despite turning the ball over `4` more times, and our model, which hates turnovers, gave that win a probability of `0.045`. Squared error charges it `1.82`. Cross-entropy charges it `3.11`.



Squared error against cross-entropy for one game: with two classes, the squared distance between $\hat{y}$ and y is $2(1 - p)^2$, so it can never go past `2`.

Cross-entropy has no ceiling, so **the more confidently wrong a prediction is, the more it costs**. That matters for our model: it gets `20` games wrong, and those `20` games make up `57%` of its total loss. An LLM sure of the wrong next word is the same problem with more classes.

Loss over a Dataset

So far we have one loss per game. To score the model on all of them we take the average over the N games:

averaged over N games, summed over the classes

[2.3] $$\mathcal{L}_{ce} = -\frac{1}{N}\sum_{n=1}^{N}\sum_{i} y_{n,i} \cdot \log(\hat{y}_{n,i})$$

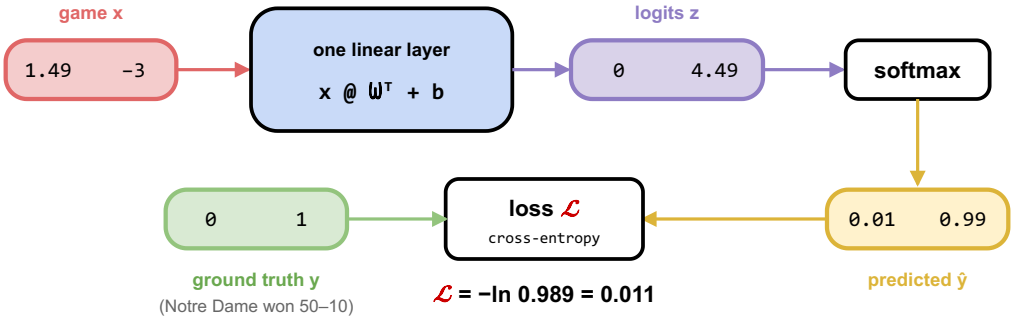
We average rather than add so the number doesn't grow just because we have more games, which lets us compare a loss on `141` games with a loss on `10`. For our hand-picked model the average is `0.341`.

On its own `0.341` doesn't mean much, so it helps to have something to compare it with. Set every weight to `0` and every logit is `0`, every game is a coin flip, and the loss is `ln 2 = 0.693` on every single game. A slightly smarter model that knows only that Notre Dame wins about three games in four (`106` of `141`) and says `[0.25, 0.75]` for every game gets `0.560`.

Our model, which actually looks at yards and turnovers, beats both. That's the use of a loss: **it puts every model, however it was made, on the same scale**.

The Whole Pipeline

Put together, here is everything that happens to one game on its way to a loss:



Every box is a function we can write down, so we can also write the whole thing as one function. Substituting the layer and softmax into cross-entropy gives

[2.4] the loss in terms of the games and the parameters

$$\mathcal{L}_{ce}(X, y;\ W, b) = -\frac{1}{N}\sum_{n=1}^{N}\sum_{i} y_{n,i} \cdot \log\left(\mathrm{softmax}(x_n W^\top + b)_i\right)$$

It looks a lot worse than it is. Reading from the inside out, it's the layer, then softmax, then the log of the probability on the right class, then the average. What matters is what it depends on: **the inputs, the ground truth and the parameters**, and nothing else. Writing a network down like this is what will let us train one.

Cross-Entropy in PyTorch

Here is the loss on all 141 games, once by hand (one-hot y, softmax, log, sum over the classes, average over the games) and once with PyTorch's own function:

```python
import torch
import torch.nn.functional as F
import pandas as pd

games = pd.read_csv("nd_football.csv")
cols = ["yard_diff", "turnover_diff"]
X = torch.tensor(games[cols].values,
                 dtype=torch.float32)
X[:, 0] = X[:, 0] / 100
y = torch.tensor(games["win"].values)

W = torch.tensor([[0., 0.], [1., -1.]])
b = torch.zeros(2)
logits = X @ W.T + b
y_hat = F.softmax(logits, dim=1)

Y = F.one_hot(y, 2)            # [0, 1] for a win
per_game = -(Y * torch.log(y_hat)).sum(dim=1)
print("by hand:", per_game.mean())
print("PyTorch:", F.cross_entropy(logits, y))
```

OUTPUT

```
by hand: tensor(0.3410)
PyTorch: tensor(0.3410)
```

Notice that `F.cross_entropy` takes the **logits**, not the probabilities (it does the softmax itself), and the class indices y rather than one-hot vectors. Handing it `y_hat` by mistake is a classic bug: it runs without complaint and gives the wrong number.

Same Picks, Different Loss

Here's a test of whether the loss sees something accuracy can't. Multiply every weight by the same positive number `s`. Every logit gets multiplied by `s` too, so the biggest logit stays the biggest and **every single prediction stays the same**:

```
import torch
import torch.nn.functional as F
import pandas as pd

games = pd.read_csv("nd_football.csv")
cols = ["yard_diff", "turnover_diff"]
X = torch.tensor(games[cols].values,
                 dtype=torch.float32)
X[:, 0] = X[:, 0] / 100
y = torch.tensor(games["win"].values)
W = torch.tensor([[0., 0.], [1., -1.]])

picks = (X @ W.T).argmax(dim=1)
for s in [0.5, 1.0, 3.0]:
    logits = X @ (s * W).T
    same = (logits.argmax(dim=1) == picks).all()
    loss = F.cross_entropy(logits, y)
    print(f"s = {s}: same picks {same}, loss {loss:.3f}")
```

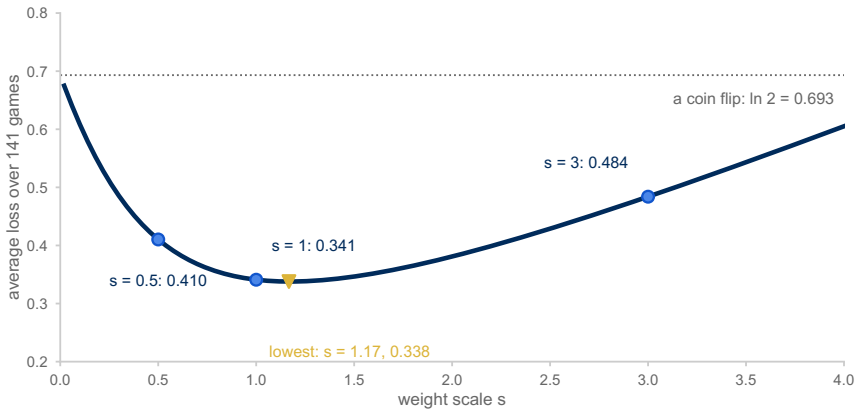
OUTPUT

```
s = 0.5: same picks tensor(True), loss 0.410
s = 1.0: same picks tensor(True), loss 0.341
s = 3.0: same picks tensor(True), loss 0.484
```

All three models get the same `121` games right, so their accuracy is identical. Their losses aren't. Halving the weights makes the model timid (Navy drops from `0.989` to `0.904`), and tripling them makes it so bold that its `20` wrong games get very expensive.

Loss as a Function of the Parameters

If we hold the games fixed, the only thing left that can change the loss is the parameters. So instead of three values of `s`, let's try all of them:



The loss as the weights grow: every point on the curve is a whole model, scored on all `141` games. Its accuracy is `121` of `141` everywhere except `s = 0`.

At `s = 0` every game is a coin flip (`0.693`). The curve falls to its lowest point, `0.338` at `s = 1.17`, a little better than our hand-picked weights, and then climbs again as the model gets overconfident. **Somewhere there is a best setting, and the loss tells us where**.

Unfortunately, `s` is only one direction to move in. Our model has six parameters (four weights in `W` and two biases in `b`), and they can all move separately. Finding the bottom of the loss in all six at once is called **optimization**.

Cross-Entropy in irishQwen

Nothing about cross-entropy is specific to football. Here is the loss function from irishQwen, the model we're building this semester, run on two of our games (Navy, and the Syracuse win the model hated):

```python
import torch
import torch.nn.functional as F

def cross_entropy(Y_hat, Y):
    # irishQwen's loss: Y is one-hot, Y_hat softmaxed
    log_probs = torch.log(Y_hat.clamp_min(1e-12))
    return -(Y * log_probs).sum(dim=-1).mean()

# 2012 Navy and 2014 Syracuse
logits = torch.tensor([[0., 4.49],
                       [0., -3.06]])
Y = torch.tensor([[0., 1.],    # won 50-10
                  [0., 1.]])   # won 31-15
Y_hat = F.softmax(logits, dim=-1)
print("Y_hat:", Y_hat)
print("loss:", cross_entropy(Y_hat, Y))
```

OUTPUT

```
Y_hat: tensor([[0.0111, 0.9889],
        [0.9552, 0.0448]])
loss: tensor(1.5585)
```

It's the equation from the Cross-Entropy page line for line: multiply the one-hot `Y` by the log of the probabilities, add up over the classes, average over the examples (`0.011` and `3.106` average to `1.558`). The `clamp_min` just keeps a probability of exactly `0` from becoming an infinite loss. (The real one also skips padding, which we'll meet later.)

Next-Token Prediction

irishQwen reads Shakespeare and, at every position, predicts what comes next. Its classes are the `512` **tokens** in its vocabulary (pieces of text, which get chapters of their own soon), so its logits are `<1, 512>`, softmax turns them into `512` probabilities, and the ground truth is a one-hot vector with its `1` on the token that actually came next.

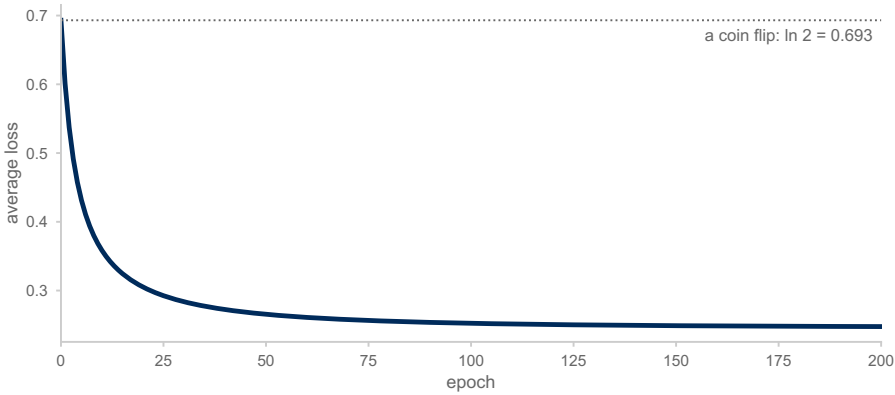
Take Hamlet's "To be, or not to be". After the token "`to`" the right answer is "` be`", and the loss depends only on how much probability a model put there:

A model that knows…	p(" be")	loss	average over the whole text
nothing (every token `1/512`)	`0.0020`	`6.24`	`6.24`
how common each token is	`0.0051`	`5.27`	`5.27`
what usually follows "`to`"	`0.056`	`2.88`	`3.58`

The last column is the average loss over all `2,526,266` tokens of irishQwen's training text (the last two models were built by counting them). A model that knows nothing scores `ln 512 = 6.24` on every token, just as our all-zeros football model scored `ln 2` on every game. **Training irishQwen is pushing that number down**, and the further down it goes, the less surprised the model is by Shakespeare.

Training a Model

Training a model lets it practice over and over on the same set of data, ideally improving every time. It improves by **updating its parameters so the loss goes down**, and one pass over every example is called an **epoch**.



A real training run: our football model, started with every parameter at `0` and updated once per epoch.

Starting from all zeros the loss is the coin flip's `0.693`, and after `200` epochs it's down to `0.248`, with `128` of the `141` games right. That is better than anything we picked by hand, and nobody chose a single weight.

What we skipped is the interesting part. Something has to look at the loss and decide how to change each parameter, and that something is called the **optimizer**. It only works because the loss is a function of the parameters that we can write down, and how it uses that function is the next chapter.